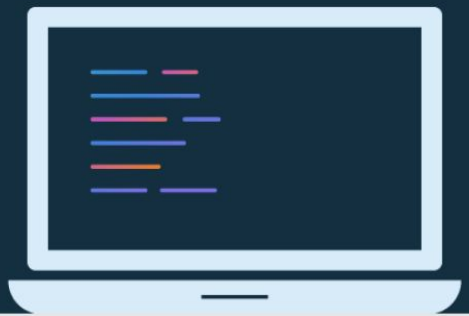


# Lesson 9: App architecture (persistence)




*This work is licensed under the Apache 2 license.*

# About this lesson

Lesson 9: App architecture (persistence)

- Storing data
- Room persistence library
- Asynchronous programming
- Coroutines
- Testing databases
- Summary


This work is licensed under the Apache 2 license.

# Storing data


*This work is licensed under the Apache 2 license.*

# Ways to store data in an Android app

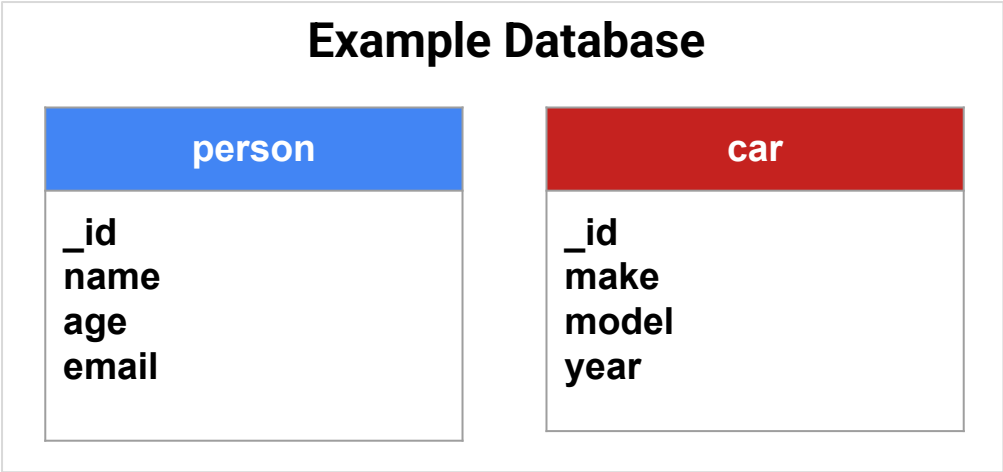
- App-specific storage
- Shared storage (files to be shared with other apps)
- Preferences
- Databases


*This work is licensed under the Apache 2 license.*

# What is a database?

Collection of structured data that can be easily accessed, searched, and organized, consisting of:

- Tables
- Rows
- Columns





This work is licensed under the Apache 2 license.

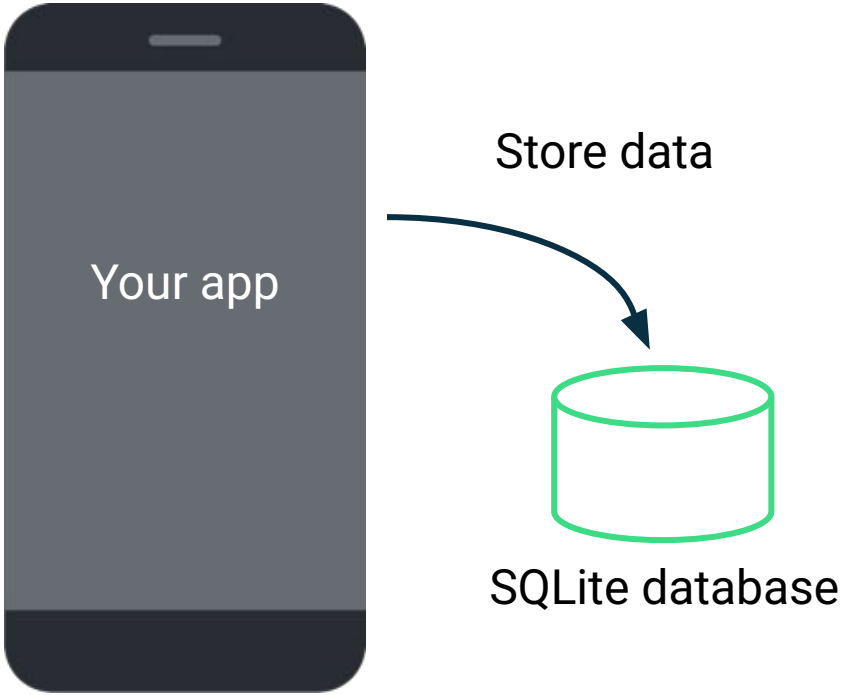
# Structured Query Language (SQL)

Use SQL to access and modify a relational database.

- Create new tables
- Query for data
- Insert new data
- Update data
- Delete data


*This work is licensed under the Apache 2 license.*

# SQLite in Android





This work is licensed under the Apache 2 license.

# Example SQLite commands

**C**reate

```
INSERT INTO colors VALUES ("red", "#FF0000");
```

**R**ead

```
SELECT * from colors;
```

**U**pdate

```
UPDATE colors SET hex="#DD0000" WHERE name="red";
```

**D**elete

```
DELETE FROM colors WHERE name = "red";
```


*This work is licensed under the Apache 2 license.*

# Interacting directly with a database

- No compile-time verification of raw SQL queries
- Need lots of boilerplate code to convert between SQL queries ⟷ data objects


This work is licensed under the Apache 2 license.

# Room persistence library


*This work is licensed under the Apache 2 license.*

# Add Gradle dependencies

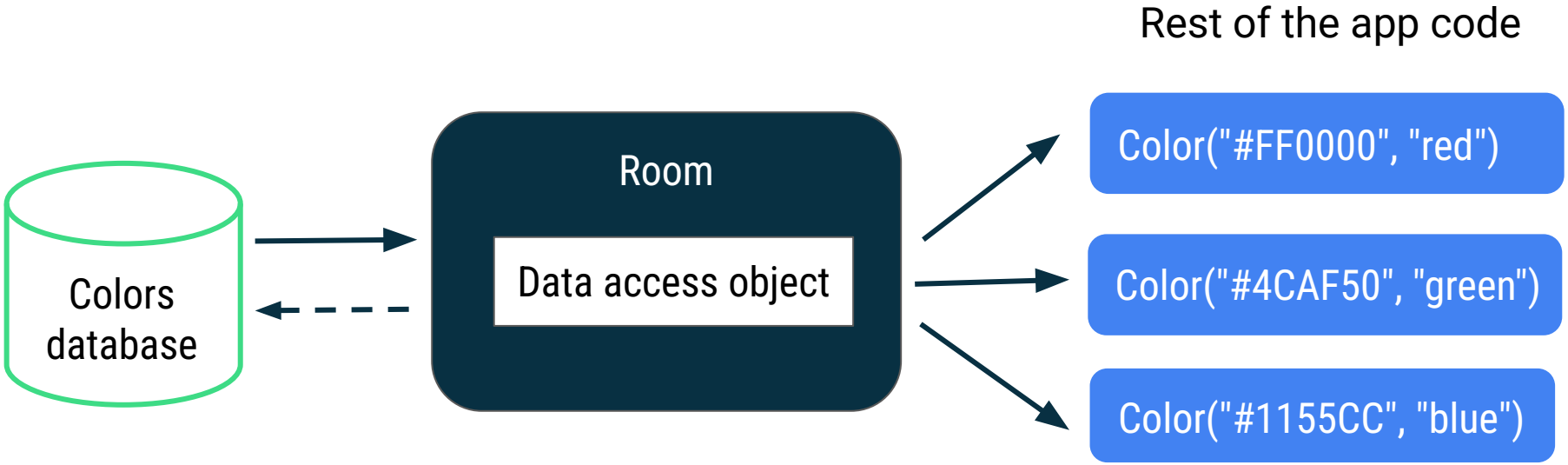
```
dependencies {
    implementation "androidx.room:room-runtime:$room_version"
    kapt "androidx.room:room-compiler:$room_version"

    // Kotlin Extensions and Coroutines support for Room
    implementation "androidx.room:room-ktx:$room_version"

    // Test helpers
    testImplementation "androidx.room:room-testing:$room_version"
}
```


*This work is licensed under the Apache 2 license.*

# Room

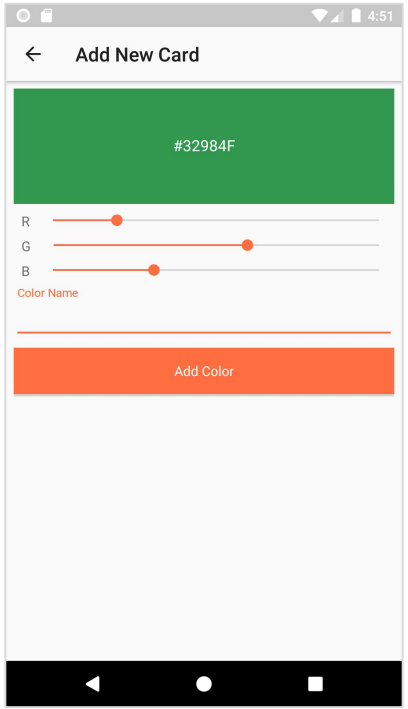


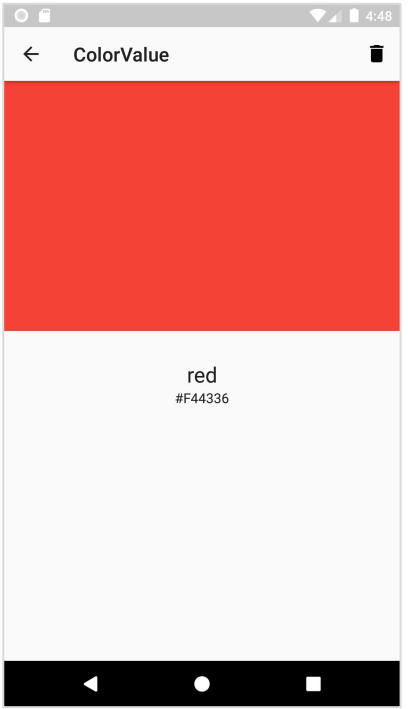


*This work is licensed under the Apache 2 license.*

# ColorValue app





*This work is licensed under the Apache 2 license.*

# Room

- Entity `Color`
- DAO `ColorDao`
- Database `ColorDatabase`


*This work is licensed under the Apache 2 license.*

# Color class

```
data class Color {
    val hex: String,
    val name: String
}
```


*This work is licensed under the Apache 2 license.*

# Annotations

- Provide extra information to the compiler

  `@Entity` marks entity class, `@Dao` for DAO, `@Database` for database

- Can take parameters

```
@Entity(tableName = "colors")
```

- Can autogenerate code for you


This work is licensed under the Apache 2 license.

# Entity

Class that maps to a SQLite database table

- `@Entity`
- `@PrimaryKey`
- `@ColumnInfo`


*This work is licensed under the Apache 2 license.*

# Example entity

```
@Entity(tableName = "colors")
data class Color {
    @PrimaryKey(autoGenerate = true) val _id: Int,
    @ColumnInfo(name = "hex_color") val hex: String,
    val name: String
}
```





This work is licensed under the Apache 2 license.

# Data access object (DAO)

Work with DAO classes instead of accessing database directly:

- Define database interactions in the DAO.
- Declare DAO as an interface or abstract class.
- Room creates DAO implementation at compile time.
- Room verifies all of your DAO queries at compile-time.


*This work is licensed under the Apache 2 license.*

# Example DAO

```
@Dao
interface ColorDao {

    @Query("SELECT * FROM colors")
    fun getAll(): Array<Color>

    @Insert
    fun insert(vararg color: Color)

    @Update
    fun update(color: Color)

    @Delete
    fun delete(color: Color)
```


*This work is licensed under the Apache 2 license.*

# Query

```
@Dao
interface ColorDao {

    @Query("SELECT * FROM colors")
    fun getAll(): Array<Color>

    @Query("SELECT * FROM colors WHERE name = :name")
    fun getColorByName(name: String): LiveData<Color>

    @Query("SELECT * FROM colors WHERE hex_color = :hex")
    fun getColorByHex(hex: String): LiveData<Color>
```


This work is licensed under the Apache 2 license.

# Insert

```
@Dao
interface ColorDao {
    ...

    @Insert
    fun insert(vararg color: Color)

    ...
}
```


This work is licensed under the Apache 2 license.

# Update

```
@Dao
interface ColorDao {
    ...

    @Update
    fun update(color: Color)

    ...
}
```


*This work is licensed under the Apache 2 license.*

# Delete

```
@Dao
interface ColorDao {
    ...

    @Delete
    fun delete(color: Color)

    ...
}
```


*This work is licensed under the Apache 2 license.*

# Create a Room database

- Annotate class with `@Database` and include list of entities:

```
@Database(entities = [Color::class], version = 1)
```

- Declare abstract class that extends `RoomDatabase`:

```
abstract class ColorDatabase : RoomDatabase() {
```

  - Declare abstract method with no args that returns the DAO:

```
abstract fun colorDao(): ColorDao
```


This work is licensed under the Apache 2 license.

# Example Room database

```
@Database(entities = [Color::class], version = 1)
abstract class ColorDatabase : RoomDatabase() {
    abstract fun colorDao(): ColorDao
    companion object {
        @Volatile
        private var INSTANCE: ColorDatabase? = null
        fun getInstance(context: Context): ColorDatabase {
            ...
        }
    }
    ...
```


*This work is licensed under the Apache 2 license.*

# Create database instance

```
fun getInstance(context: Context): ColorDatabase {
    return INSTANCE ?: synchronized(this) {
        INSTANCE ?: Room.databaseBuilder(
            context.applicationContext,
            ColorDatabase::class.java, "color_database"
        )
        .fallbackToDestructiveMigration()
        .build()
        .also { INSTANCE = it }
    }
}
```


*This work is licensed under the Apache 2 license.*

# Get and use a DAO

Get the DAO from the database:

```
val colorDao = ColorDatabase.getInstance(application).colorDao()
```

Create new Color and use DAO to insert it into database:

```
val newColor = Color(hex = "#6200EE", name = "purple")

colorDao.insert(newColor)
```


This work is licensed under the Apache 2 license.

# Asynchronous programming


*This work is licensed under the Apache 2 license.*

# Long-running tasks

- Download information
- Sync with a server
- Write to a file
- Heavy computation
- Read from, or write to, a database

You shouldn't execute long-running tasks on the main thread or your app will be unresponsive to the user.

These kind of tasks need to be executed asynchronously.


*This work is licensed under the Apache 2 license.*

# Need for async programming

- Limited time to do tasks and remain responsive
  - UI frames must be rendered in <16 ms to achieve a 60 fps refresh rate. If your app's UI rendering is slower than that, the system will skip frames causing stutter.
- Balanced with the need to execute long-running tasks
- Control over how and where tasks are executed


This work is licensed under the Apache 2 license.

# Async programming on Android

- Threading
- Callbacks
- Plus many other options

What is the recommended way?


This work is licensed under the Apache 2 license.

# Coroutines


*This work is licensed under the Apache 2 license.*

# Coroutines

- A coroutine is a concurrency design pattern
- Keep your app responsive while managing long-running tasks.
- Simplify asynchronous code in your Android app.
- Write code in sequential way
- Handle exceptions with `try/catch` block


*This work is licensed under the Apache 2 license.*

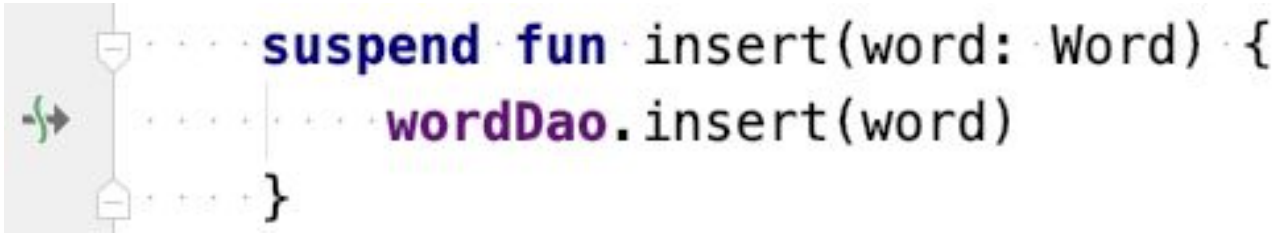
# Benefits of coroutines

- Lightweight
- Fewer memory leaks
- Built-in cancellation support
- Jetpack integration


*This work is licensed under the Apache 2 license.*

# Suspend functions

- Add `suspend` modifier
- Must be called by other `suspend` functions or coroutines





This work is licensed under the Apache 2 license.

# Suspend and resume

- `suspend`

  Pauses execution of c

- `resume`

  Automatically loads saved state and continues execution from the point the code was suspended

When a coroutine calls a function marked as `suspend`, instead of blocking until that function returns, it suspends execution until the result is ready and then it resumes where it left off with the result.

While it's suspended waiting for a result, it unblocks the thread that it's running on so other functions or coroutines can run.


*This work is licensed under the Apache 2 license.*

# Example




1. `fetchDocs()` 's marked as a `suspend` function.
2. `fetchDocs()` keeps track of its state and executes normally until it can no longer run when it reaches `get()`, which is also a `suspend` function.
3. Execution is handed over to `get()` which stores its state and attempts to run.
4. Eventually the tasks `get()` was waiting for return and `get()` resumes execution.
5. In this case, `get()` has no other tasks and it yields back to `fetchDocs()` so it can resume.
6. It plugs the values from the returned `get()` into its execution flow and runs to completion.


*This work is licensed under the Apache 2 license.*

# Add suspend modifier to DAO methods

```
@Dao
interface ColorDao {

    @Query("SELECT * FROM colors")
    suspend fun getAll(): Array<Color>

    @Insert
    suspend fun insert(vararg color: Color)

    @Update
    suspend fun update(color: Color)

    @Delete
    suspend fun delete(color: Color)
```

- Room has coroutines support
- **suspend** modifier enforces that methods can be only called from within a coroutine or from another suspend function.


*This work is licensed under the Apache 2 license.*

# Control where coroutines run

| Dispatcher | Description of work | Examples of work |
|---|---|---|
| `Dispatchers.Main` | UI and nonblocking (short) tasks | Updating LiveData, calling suspend functions |
| `Dispatchers.IO` | Network and disk tasks | Database, file IO |
| `Dispatchers.Default` | CPU intensive | Parsing JSON |

Using `suspend` doesn't tell Kotlin to run a function on a background thread. Coroutines can run on the main thread.


*This work is licensed under the Apache 2 license.*

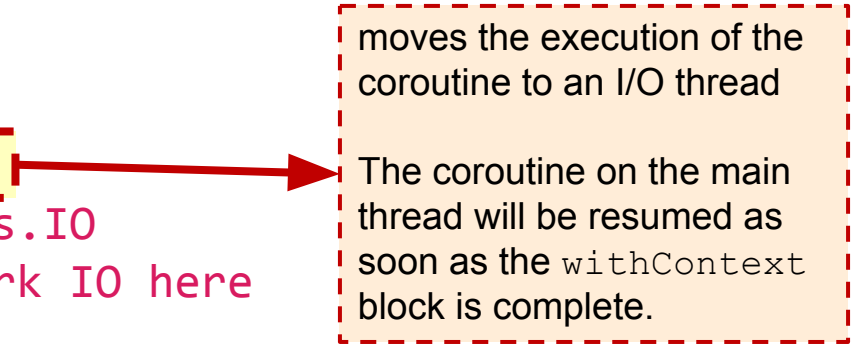
# withContext

```
suspend fun get(url: String) {

    // Start on Dispatchers.Main

    withContext(Dispatchers.IO) {
        // Switches to Dispatchers.IO
        // Perform blocking network IO here
    }

    // Returns to Dispatchers.Main
}
```





This work is licensed under the Apache 2 license.

# CoroutineScope

Coroutines must run in a `CoroutineScope`:

- Keeps track of all coroutines started in it (even suspended ones)
- Provides a way to cancel coroutines in a scope
- Provides a bridge between regular functions and co

Examples: **`GlobalScope`** *(defa*

`ViewModel` has `v`

`Lifecycle` has `li` `eScope`

`CoroutineScope` keeps track of your coroutines, and it can cancel the coroutines started in it. It doesn't execute your coroutines, it just makes sure you don't lose track of them.


This work is licensed under the Apache 2 license.

# How to start new coroutines?

- `launch` - no result needed

```
fun loadUI() {
    launch {
        fetchDocs()
    }
}
```

launch block starts a new coroutine in the default coroutine scope with the default dispatcher.

It can call suspend functions

- `async` - can return a result (*you can get the result with `await` keyword*)


*This work is licensed under the Apache 2 license.*

# ViewModelScope

```
class MyViewModel: ViewModel() {

    init {
        viewModelScope.launch {
            // Coroutine that will be canceled
            // when the ViewModel is cleared
        }
    }
    ...
```


*This work is licensed under the Apache 2 license.*

# Example viewModelScope

```
class ColorViewModel(val dao: ColorDao, application: Application)
    : AndroidViewModel(application) {

    fun save(color: Color) {
        viewModelScope.launch {
            colorDao.insert(color)
        }
    }


    ...
```


*This work is licensed under the Apache 2 license.*

# Testing databases


*This work is licensed under the Apache 2 license.*

# Add Gradle dependencies

```
android {
    defaultConfig {
        ...
        testInstrumentationRunner "androidx.test.runner
         .AndroidJUnitRunner"
        testInstrumentationRunnerArguments clearPackageData: 'true'
    }
}

dependencies {
    testImplementation 'junit:junit:4.12'
    androidTestImplementation 'androidx.test.ext:junit:1.1.0'
    androidTestImplementation 'androidx.test.espresso:espresso-core:3.1.1'
}
```


*This work is licensed under the Apache 2 license.*

# Testing Android code

- `@RunWith(AndroidJUnit4::class)`
- `@Before`
- `@After`
- `@Test`


*This work is licensed under the Apache 2 license.*

# Create test class

```
@RunWith(AndroidJUnit4::class)
class DatabaseTest {

    private lateinit val colorDao: ColorDao
    private lateinit val db: ColorDatabase

    private val red = Color(hex = "#FF0000", name = "red")
    private val green = Color(hex = "#00FF00", name = "green")
    private val blue = Color(hex = "#0000FF", name = "blue")

    ...
```


*This work is licensed under the Apache 2 license.*

# Create and close database for each test

In `DatabaseTest.kt`:

```
@Before
fun createDb() {
    val context: Context = ApplicationProvider.getApplicationContext()
    db = Room.inMemoryDatabaseBuilder(context, ColorDatabase::class.java)
        .allowMainThreadQueries()
        .build()
    colorDao = db.colorDao()
}

@After
@Throws(IOException::class)
fun closeDb() = db.close()
```


*This work is licensed under the Apache 2 license.*

# Test insert and retrieve from a database

In `DatabaseTest.kt`:

```
@Test
@Throws(Exception::class)
fun insertAndRetrieve() {
    colorDao.insert(red, green, blue)
    val colors = colorDao.getAll()
    assert(colors.size == 3)
}
```


*This work is licensed under the Apache 2 license.*

# Summary


*This work is licensed under the Apache 2 license.*

# Summary

In Lesson 9, you learned how to:

- Set up and configure a database using the Room library
- Use coroutines for asynchronous programming
- Use coroutines with Room
- Test a database


*This work is licensed under the Apache 2 license.*

# Learn more

- 7 Pro-tips for Room
- Room Persistence Library
- SQLite Home Page
- Save data using SQLite
- Coroutines Guide
- Dispatchers - kotlinx-coroutines-core
- Coroutines on Android (part I): Getting the background
- Coroutines on Android (part II): Getting started
- Easy Coroutines in Android: viewModelScope
- Kotlin Coroutines 101


*This work is licensed under the Apache 2 license.*

# Pathway

Practice what you've learned by completing the pathway:

Lesson 9: App architecture (persistence)




*This work is licensed under the Apache 2 license.*